Departamento de Engenharia Informática

Inteligência Artificial para Entretenimento

Projeto 1: Co-Creative Curation Pipeline on MovieLens 100K

Elementos do grupo:

Miguel Soares Francisco – Nº 2025179328 –
uc2025179328@student.uc.pt
Miguel Oliveira Pinto – Nº 2025190104 – uc2025190104@student.uc.pt
Gonçalo José Carrajola Gaio – Nº 2022224905 –
uc2022224905@student.uc.pt

Coimbra, 2025-2026

Conteúdo

1 Introdução **2**

2 Arquitetura do Sistema **2**

3 Conjunto de dados e pré-processamento **3**

4 Implementação da pipeline **4**

4.1 Configuração experimental 5

5 Resultados **5**

5.1 Comparação entre métodos base 5

5.2 Impacto do reranking com MMR 6

5.3 Sessões de personalização com EMA 8

5.4 Curvas de treino (modo completo) 8

6 Testes adicionais **9**

6.1 Remoção do MMR 9

6.2 Remoção da feature de popularidade no Pairwise LTR 10

7 Conclusão **10**

A Apêndice Teórico: Respostas T1-T5 **11**

1 Introdução

No contexto da disciplina de Inteligência Artificial para Entretenimento, os sistemas de recomendação têm um papel fundamental na seleção e organização de conteúdos relevantes para cada utilizador. Neste projeto, esse problema é abordado sob a perspetiva de *co-creative curation*, em que a recomendação é vista como um processo iterativo que combina geração de candidatos, ranking, reranking, feedback e atualização da personalização. Com base no dataset MovieLens 100K, foi desenvolvido uma pipeline completo capaz de produzir listas top-K de filmes adaptadas às preferências dos utilizadores. Para este efeito, foram implementados e comparados diferentes métodos de ranking, incluindo um baseline de popularidade, modelos de fatorização matricial com SGD e ALS, e um modelo de *Pairwise Learning-to-Rank*. Adicionalmente, foi incorporado um mecanismo de reranking com MMR para controlar a diversidade das recomendações e um mecanismo de atualização do estado do utilizador com EMA para simular sessões sucessivas de interação. A avaliação experimental foi realizada com recurso às métricas Recall@10, NDCG@10 e Diversity@10, permitindo analisar não só a relevância das recomendações, mas também o equilíbrio entre precisão, diversidade e adaptação dinâmica ao utilizador.

2 Arquitetura do Sistema

O sistema segue a sequência exigida no enunciado: geração de candidatos, ranking, reranking com controlo de diversidade, feedback e atualização do estado do utilizador. A Tabela 1 resume os principais módulos que implementam cada bloco da pipeline e a respetiva função no fluxo de execução.

Ficheiro / Módulo	Função
`src/main.py`	Orquestra toda a execução do projeto: configuração, treino dos modelos, avaliação, aplicação de MMR, sessões com EMA e criação de logs.
`src/core/config.py`	Define os principais parâmetros experimentais, hiperparâmetros dos modelos e opções de execução, incluindo o modo FAST MODE.
`src/core/structs.py`	Contém as estruturas de dados e classes auxiliares usadas para guardar resultados, variáveis globais e modelos.
`src/core/candidates.py`	Gera o conjunto de filmes candidatos para cada utilizador, excluindo os itens já observados no treino.
`src/core/utils.py`	Reúne funções auxiliares para carregamento de dados, preparação da avaliação, geração de gráficos, comparação de métodos e escrita de logs.
`src/core/metrics.py`	Implementa as métricas de avaliação usadas no projeto, nomeadamente Recall@10, NDCG@10, similaridade do cosseno e Diversity@10.
`src/rankers/popularity.py`	Implementa o baseline de popularidade, responsável por recomendar filmes com base no número de interações no treino.
`src/rankers/mf_general.py`	Contém funções gerais para os modelos de fatorização matricial, incluindo preparação dos dados, previsão de ratings, recomendação e cálculo de RMSE.
`src/rankers/mf_sgd.py`	Implementa o treino do modelo de Matrix Factorization com SGD, incluindo fatores latentes, biases e histórico de treino.
`src/rankers/mf_als.py`	Implementa o treino do modelo de Matrix Factorization com ALS, atualizando alternadamente fatores latentes e biases.
`src/rankers/pairwise_ltr.py`	Implementa o modelo Pairwise Learning-to-Rank, incluindo criação dos pares de treino, construção das features, treino e recomendação.
`src/reranker/mmr.py`	Implementa o reranking com MMR para aumentar a diversidade das listas recomendadas, equilibrando relevância e diversidade.
`src/personalization/ema.py`	Implementa a atualização dinâmica do estado do utilizador com EMA e a simulação de sessões iterativas de recomendação.

Tabela 1: Descrição resumida dos principais ficheiros do projeto.

3 Conjunto de dados e pré-processamento

O sistema foi desenvolvido com base no *MovieLens 100K*, com avaliações explícitas e metadados de filmes (título e géneros). Estes géneros são essenciais para a medida de diversidade e para a similaridade no MMR. A Tabela 2 sintetiza as decisões principais e os números usados no pré-processamento.

Aspeto	Decisão/Resumo
Dataset	MovieLens 100K; 100 000 avaliações, 943 utilizadores, 1682 filmes; tabela de itens com 24 colunas.
Split	Divisão aleatória 80/20: 80 000 interações em treino e 20 000 em teste.
Relevância	$G_u = \{i : (u,i) \in \Omega_{\text{test}} \wedge r_{ui} \geq 4\}$; itens do treino excluídos das recomendações.
Utilizadores elegíveis	920 com pelo menos um item relevante; avaliação em 300 utilizadores (modo rápido).
Representação MF	Mapeamento para índices consecutivos; dados em tuplos (u,i,r); estruturas por utilizador/item para ALS.
LTR pairwise	46 409 pares de preferência (i^+, i^-) gerados a partir do treino.
Diversidade/MMR	Vetores binários de géneros; similaridade por cosseno.

Tabela 2: Resumo do conjunto de dados e pré-processamento.

Em termos operacionais, as avaliações foram carregadas e divididas em treino e teste, aplicando a definição de relevância do enunciado e excluindo itens já observados no treino durante a recomendação. Para os modelos de fatorização matricial, a indexação consecutiva permitiu construir as matrizes latentes e as estruturas auxiliares necessárias ao SGD e ao ALS. Para o *pairwise LTR*, geraram-se pares de preferência por utilizador, alinhando o treino com a tarefa de ordenação. Finalmente, os vetores de género foram preparados para suportar tanto o MMR como o cálculo de Diversity@10.

4 Implementação da pipeline

O fluxo completo de *co-creative curation* está concentrado em `main.py` e segue o percurso exigido: geração de candidatos, ranking, reranking, feedback e atualização do estado do utilizador. O pipeline avalia primeiro o baseline de popularidade, treina e avalia os modelos de fatorização matricial e o *pairwise LTR*, aplica MMR sobre os top-M candidatos e, por fim, simula sessões com EMA.

A Tabela 3 sintetiza como cada etapa foi concretizada. Em todas as fases, os resultados são registados em JSONL com identificador interno do utilizador, top-K, métricas e hiperparâmetros; nas sessões EMA, inclui-se o item escolhido e um resumo do estado atualizado.

Etapa	Implementação (`main.py`)
Candidatos	$C(u)$ inclui todos os itens não avaliados no treino; usado por todos os rankers.
Popularidade	Avaliação do baseline com ordenação por contagem global.
MF-SGD / MF-ALS	Treino com vieses e avaliação com Recall@10, NDCG@10 e Diversity@10.
Pairwise LTR	Pares (i^+, i^-) gerados no treino; modelo Bradley–Terry com $f(u,i) = [s_{MF}, b_u, b_i, pop(i), 1]$.
MMR	Reranking dos top-M com $\alpha \in \{0.1, 0.4, 0.7\}$ e similaridade por cosseno dos géneros.
EMA	Sessões com 5 rondas, escolha heurística e atualização $u_{t+1} = \frac{(1-\rho)u_t + \rho v_i}{\|(1-\rho)u_t + \rho v_i\|}$, usando $\rho \in \{0.1, 0.3\}$.
Logs	JSONL por método e por sessão, com top-K, métricas e hiperparâmetros.

Tabela 3: Resumo da implementação da pipeline em `main.py`.

O modo rápido controla o número de utilizadores avaliados e os principais hiperparâmetros (dimensão latente, épocas/iterações e M do MMR), garantindo execução completa em tempo reduzido, conforme requerido.

4.1 Configuração experimental

Para clarificar as definições usadas, a Tabela 4 resume os parâmetros mais relevantes no modo rápido e no modo completo. Os restantes hiperparâmetros (por exemplo, $K = 10$, α do MMR e ρ do EMA) são mantidos constantes entre modos, conforme indicado no enunciado.

Parâmetro	**Modo rápido**	**Modo completo**
Utilizadores avaliados	300	Todos os elegíveis
Dimensão latente (MF)	20	50
Épocas MF-SGD	10	25
Iterações MF-ALS	8	15
Top-M para MMR	80	200
Top-K	10	10
α do MMR	$\{0.1, 0.4, 0.7\}$	$\{0.1, 0.4, 0.7\}$
ρ do EMA	$\{0.1, 0.3\}$	$\{0.1, 0.3\}$
Pares por utilizador (LTR)	50	50
Épocas LTR	5	5

Tabela 4: Resumo das configurações de execução.

Nota: o modo rápido é controlado pela *flag* `FAST_MODE` no ficheiro `config.py`.

5 Resultados

Nesta secção apresentam-se os principais resultados obtidos para os diferentes métodos implementados na pipeline de recomendação, considerando as métricas Recall@10, NDCG@10 e Diversity@10. Os valores reportados correspondem ao modo completo (`FAST_MODE` desativado), com avaliação sobre todos os utilizadores elegíveis.

5.1 Comparação entre métodos base

A Tabela 5 resume os resultados médios obtidos pelos quatro métodos principais avaliados: baseline de popularidade, fatorização matricial com SGD, fatorização matricial com ALS e modelo *pairwise learning-to-rank* (E1). No modo completo, o baseline de popularidade apresenta os melhores valores de Recall@10 (0.1372) e NDCG@10 (0.1764), o que indica que a popularidade global continua a ser um sinal forte neste dataset.

O MF-SGD melhora relativamente ao MF-ALS, mas continua abaixo do baseline em relevância; o MF-ALS mantém o pior desempenho em Recall@10 e NDCG@10. O *pairwise LTR* apresentou valores inferiores aos do baseline (Recall@10 de 0.0314 e NDCG@10 de 0.0470), mantendo diversidade semelhante. Este comportamento reforça a necessidade de analisar o impacto das features e dos testes adicionais.

Uma interpretação plausível para o desempenho do LTR é a escala heterogénea das features (por exemplo, a popularidade em contagens absolutas) e o facto de a previsão MF estar limitada ao intervalo

de ratings. Sem normalização explícita, o modelo linear pode favorecer sinais dominantes, o que ajuda a explicar a melhoria observada quando a popularidade é removida.

Tabela 5: Comparação dos métodos base na avaliação offline.

Método	**Recall@10**	**NDCG@10**	**Diversity@10**
Popularidade	0.1372	0.1764	0.7517
MF-SGD	0.0506	0.0805	0.7338
MF-ALS	0.0196	0.0297	0.7086
Pairwise LTR	0.0314	0.0470	0.7511

De forma geral, os resultados indicam que otimizar diretamente para ordenação relativa entre itens foi mais eficaz do que usar exclusivamente modelos de fatorização matricial treinados para previsão de ratings. O desempenho do modelo pairwise LTR confirma, assim, a importância de alinhar o objetivo de treino com a tarefa final de ranking.

5.2 Impacto do reranking com MMR

Um dos requisitos experimentais do projeto (E2) consiste em apresentar um gráfico do compromisso entre relevância e diversidade, medido em termos de NDCG@10 e Diversity@10, para diferentes valores do parâmetro $\alpha \in \{0.1, 0.4, 0.7\}$, aplicando o método MMR pelo menos a um baseline de fatorização matricial. Para satisfazer este requisito, foi analisado o efeito do reranking com MMR sobre o modelo MF-SGD, representando graficamente a relação entre NDCG@10 e Diversity@10 para os diferentes valores de α.

Após a avaliação dos rankers base, o método MMR foi aplicado para introduzir controlo explícito da diversidade nas listas finais. A Tabela 6 apresenta os resultados médios para diferentes valores de α, usando como base os modelos MF-SGD, MF-ALS e Pairwise LTR, enquanto a Figura 1 ilustra graficamente o trade-off obtido no caso do MF-SGD.

No caso do MF-SGD, o reranking com MMR aumentou significativamente a diversidade: de 0.7338 sem reranking para 0.7957, 0.9304 e 0.9909 para $\alpha = 0.1$, 0.4 e 0.7, respetivamente. Em paralelo, os valores de NDCG@10 passaram de 0.0805 (sem MMR) para 0.0790, 0.0718 e 0.0579 à medida que a diversidade aumentou. A Figura 1 ilustra este compromisso, com $\alpha = 0.4$ a oferecer um equilíbrio entre diversidade elevada e perda moderada de relevância.

Para o MF-ALS, o MMR também aumentou muito a diversidade, elevando-a para valores próximos de 0.96. No entanto, como o modelo base já apresentava fraca qualidade de ranking, o reranking não foi suficiente para recuperar relevância. Já no caso do Pairwise LTR, observou-se um comportamento mais equilibrado: a diversidade aumentou progressivamente com α, enquanto Recall@10 e NDCG@10 sofreram apenas uma ligeira degradação para valores mais elevados deste parâmetro. A Figura 2 mostra estes trade-offs adicionais e reforça o compromisso clássico entre relevância e diversidade.

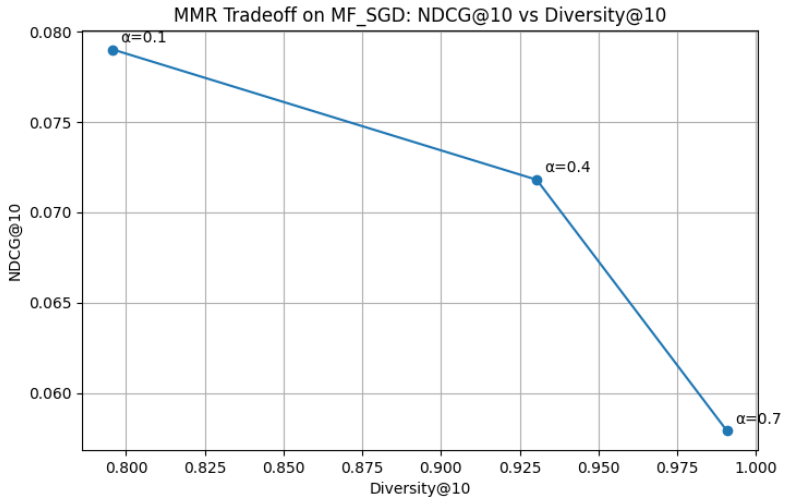

Figura 1: Trade-off entre NDCG@10 e Diversity@10 para o reranking com MMR aplicado ao modelo MF-SGD, considerando $\alpha \in \{0.1, 0.4, 0.7\}$.

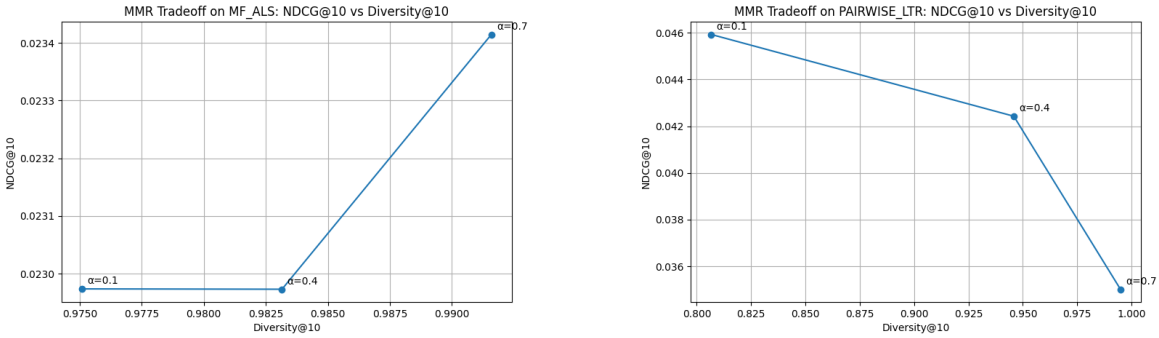

Figura 2: Trade-off do MMR nos modelos MF-ALS e Pairwise LTR (modo completo).

Tabela 6: Resultados do reranking com MMR para diferentes valores de α.

Modelo Base	α	**Recall@10**	**NDCG@10**	**Diversity@10**
MF-SGD	0.1	0.0492	0.0790	0.7957
MF-SGD	0.4	0.0444	0.0718	0.9304
MF-SGD	0.7	0.0295	0.0579	0.9909
MF-ALS	0.1	0.0120	0.0230	0.9751
MF-ALS	0.4	0.0120	0.0230	0.9832
MF-ALS	0.7	0.0123	0.0234	0.9916
Pairwise LTR	0.1	0.0302	0.0459	0.8066
Pairwise LTR	0.4	0.0281	0.0424	0.9459
Pairwise LTR	0.7	0.0191	0.0350	0.9949

Entre os resultados obtidos, destaca-se particularmente o Pairwise LTR com MMR e $\alpha = 0.4$, que preserva um NDCG@10 muito próximo do melhor valor absoluto observado, ao mesmo tempo que aumenta claramente a diversidade da lista. Este comportamento mostra que a fase de reranking pode acrescentar

valor mesmo quando o ranker base já apresenta boa qualidade.

5.3 Sessões de personalização com EMA

Para além da avaliação estática, foi ainda demonstrada a componente dinâmica de personalização através de sessões sucessivas com atualização por EMA. Foram realizadas sessões com 3 utilizadores, 5 rondas por sessão e comparação entre $\rho = 0.1$ e $\rho = 0.3$. Em cada ronda, o sistema produziu uma lista recomendada, simulou a escolha de um item e atualizou o vetor de estado do utilizador com base no item selecionado.

Os resultados qualitativos mostram alterações graduais das listas ao longo das rondas. No MF-SGD, a sobreposição média entre listas consecutivas manteve-se elevada para $\rho = 0.1$ (entre 0.768 e 0.818), enquanto que, com $\rho = 0.3$ desceu para intervalos entre 0.624 e 0.725, indicando maior sensibilidade ao feedback recente. No MF-ALS, observou-se padrão semelhante: $\rho = 0.1$ oscilou entre 0.674 e 0.818, e $\rho = 0.3$ entre 0.465 e 0.632. Estas tendências são coerentes com a interpretação de ρ como taxa de atualização.

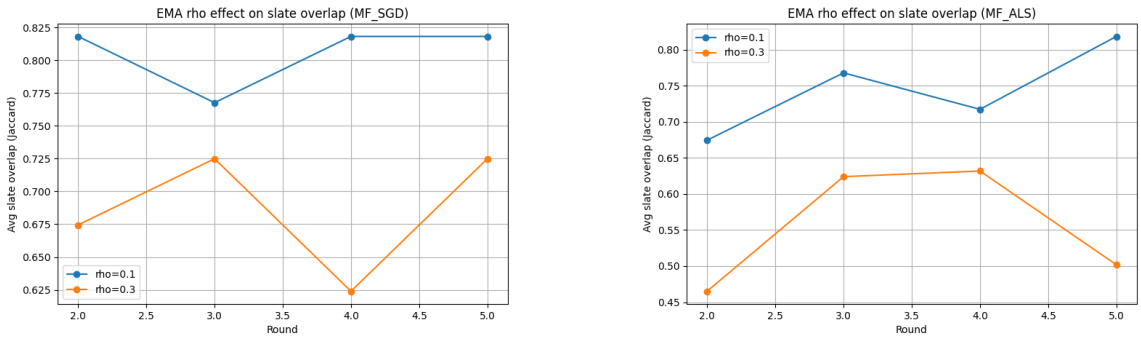

Figura 3: Efeito de ρ na sobreposição média entre listas consecutivas (Jaccard) em sessões EMA com MF-SGD e MF-ALS.

Estes resultados confirmam que o mecanismo de EMA implementado é capaz de introduzir adaptação temporal na pipeline, permitindo que o sistema reaja ao feedback do utilizador ao longo de uma sessão, em vez de produzir sempre listas estáticas.

5.4 Curvas de treino (modo completo)

Para complementar a análise, apresentam-se as curvas de treino dos modelos MF e LTR no modo completo. Observa-se uma descida consistente do RMSE nos modelos MF e uma estabilização da perda no LTR, indicando convergência do processo de treino.

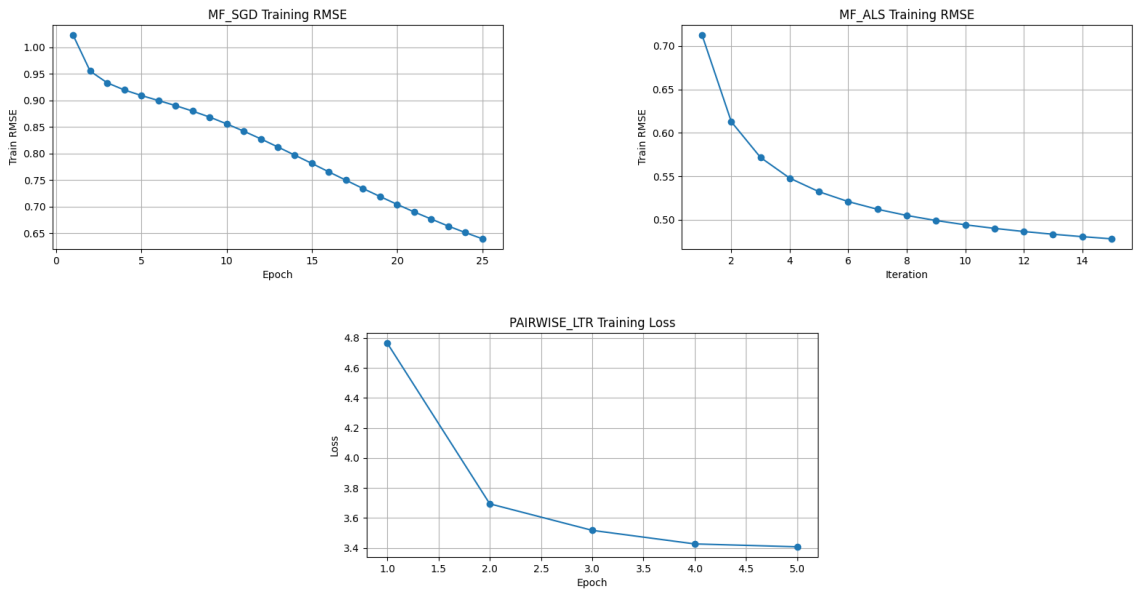

Figura 4: Curvas de treino no modo completo: RMSE do MF-SGD e MF-ALS e perda do Pairwise LTR.

6 Testes adicionais

De acordo com o enunciado do projeto (E3), foram considerados estudos com testes adicionais com o objetivo de perceber o contributo de componentes específicas da pipeline, nomeadamente o mecanismo de reranking com MMR e a utilização da feature de popularidade no modelo *pairwise learning-to-rank*.

6.1 Remoção do MMR

Comparamos o desempenho dos modelos sem reranking e com MMR para três níveis de α (0.1, 0.4 e 0.7). O padrão é claro: o MMR aumenta substancialmente a diversidade, mas reduz a relevância (Recall@10 e NDCG@10), especialmente para $\alpha = 0.7$.

Modelo	**Configuração**	**Recall@10**	**NDCG@10**	**Diversity@10**
MF-SGD	Sem MMR	0.0506	0.0805	0.7338
MF-SGD	MMR ($\alpha = 0.1$)	0.0492	0.0790	0.7957
MF-SGD	MMR ($\alpha = 0.4$)	0.0444	0.0718	0.9304
MF-SGD	MMR ($\alpha = 0.7$)	0.0295	0.0579	0.9909
MF-ALS	Sem MMR	0.0196	0.0297	0.7086
MF-ALS	MMR ($\alpha = 0.1$)	0.0120	0.0230	0.9751
MF-ALS	MMR ($\alpha = 0.4$)	0.0120	0.0230	0.9832
MF-ALS	MMR ($\alpha = 0.7$)	0.0123	0.0234	0.9916
Pairwise LTR	Sem MMR	0.0314	0.0470	0.7511
Pairwise LTR	MMR ($\alpha = 0.1$)	0.0302	0.0459	0.8066
Pairwise LTR	MMR ($\alpha = 0.4$)	0.0281	0.0424	0.9459
Pairwise LTR	MMR ($\alpha = 0.7$)	0.0191	0.0350	0.9949

Tabela 7: Impacto do MMR nas métricas de ranking e diversidade.

Em termos práticos, $\alpha = 0.4$ tende a ser um *sweet spot*, pois aumenta bastante a diversidade sem degradar demasiado a relevância.

6.2 Remoção da feature de popularidade no Pairwise LTR

Este teste compara o modelo pairwise LTR com e sem a feature de popularidade. Os resultados mostram um aumento de relevância ao remover a popularidade, acompanhado de uma ligeira redução da diversidade.

Configuração	**Recall@10**	**NDCG@10**	**Diversity@10**
Com popularidade	0.0314	0.0470	0.7511
Sem popularidade	0.0520	0.0857	0.7219

Tabela 8: Ablation do Pairwise LTR: efeito da feature de popularidade.

7 Conclusão

Este trabalho implementou uma pipeline completa de *co-creative curation* e permitiu comparar baselines simples com modelos mais avançados, avaliando simultaneamente a relevância, a diversidade e a adaptação temporal. Os resultados mostram que a popularidade continua a ser um sinal forte neste dataset, que o reranking por MMR melhora claramente a diversidade com um custo controlável em relevância, e que o EMA introduz uma dinâmica coerente de atualização do perfil do utilizador. Os testes adicionais confirmam ainda que o desempenho do *pairwise LTR* é sensível às features usadas, reforçando a importância de alinhar a representação e o objetivo de treino com a tarefa de ranking.

Quanto à escala das métricas, valores de Recall@10 e NDCG@10 baixos são esperados neste cenário: o catálogo é grande, o corte é restrito a $K = 10$ e a relevância é definida por um limiar de rating, o que aumenta o número de itens relevantes por utilizador e comprime as métricas. Assim, apesar de estarem em $[0, 1]$, os valores absolutos não são diretamente comparáveis a outros contextos. O desempenho particularmente baixo do MF-ALS é plausível por estar otimizado para erro quadrático em ratings (não para ranking) e por ser sensível a hiperparâmetros e regularização; com as configurações usadas, pode subajustar e produzir ordenações menos discriminativas do que o MF-SGD. Em suma, o projeto evidencia o compromisso clássico entre precisão e variedade e sublinha que a qualidade final depende tanto do ranker base como das etapas de reranking e personalização.

A Apêndice Teórico: Respostas T1-T5

T1 (ALS closed-form update)

1. Redução a um Problema de Regressão Ridge

O objetivo da Fatoração de Matrizes (MF) é minimizar o erro quadrático entre a classificação real r_{ui} e a prevista $\hat{r}_{ui}$. A previsão é definida como:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i \tag{1}$$

Para isolar a otimização do vetor do utilizador p_u, definimos o **resíduo** e_{ui} como a parte do rating que não é explicada pelos termos fixos (μ, b_u, b_i):

$$e_{ui} = r_{ui} - (\mu + b_u + b_i) \tag{2}$$

Ao fixar os vetores dos itens Q, a função de custo local para um utilizador u com regularização L_2 torna-se:

$$\min_{p_u} \sum_{i \in \Omega(u)} (e_{ui} - p_u^\top q_i)^2 + \lambda \|p_u\|^2 \tag{3}$$

Esta expressão é a forma canónica de uma **Regressão Ridge**, onde p_u atua como o vetor de pesos, q_i como as características (features) e e_{ui} como o valor alvo.

2. Derivação das Equações Normais

Para encontrar o valor ótimo de p_u, calculamos o gradiente da função de custo $J(p_u)$ e igualamos a zero:

Passo A: Gradiente

$$\nabla_{p_u} J(p_u) = \sum_{i \in \Omega(u)} -2(e_{ui} - p_u^\top q_i) q_i + 2\lambda p_u \tag{4}$$

Passo B: Condição de Primeira Ordem Igualando a zero e simplificando o fator 2:

$$\sum_{i \in \Omega(u)} (p_u^\top q_i) q_i + \lambda p_u = \sum_{i \in \Omega(u)} e_{ui} q_i \tag{5}$$

Passo C: Formulação Matricial Reescrevendo o produto escalar $(p_u^\top q_i) q_i$ como $(q_i q_i^\top) p_u$ e utilizando a matriz Identidade I:

$$\left(\sum_{i \in \Omega(u)} q_i q_i^\top + \lambda I \right) p_u = \sum_{i \in \Omega(u)} e_{ui} q_i \tag{6}$$

Passo D: Solução de Forma Fechada Multiplicando pelo inverso da matriz resultante, obtemos a regra de atualização do ALS:

$$p_u = \left(\sum_{i \in \Omega(u)} q_i q_i^\top + \lambda I \right)^{-1} \left(\sum_{i \in \Omega(u)} e_{ui} q_i \right) \tag{7}$$

T2 (SGD updates for MF with biases)

A função de previsão para a classificação $\hat{r}_{ui}$ é definida como:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i \tag{8}$$

O objetivo é minimizar o erro quadrático com regularização ℓ_2 para cada observação $(u, i) \in \Omega$. A função de custo local J_{ui} é:

$$J_{ui} = (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2) \tag{9}$$

Definimos o erro de previsão como $\epsilon_{ui} = r_{ui} - \hat{r}_{ui}$. Para derivar as atualizações SGD, calculamos o gradiente negativo de J_{ui} em ordem a cada parâmetro.

1. Derivação dos Gradientes

Aplicando a regra da cadeia, os gradientes em relação aos parâmetros são:

- $\frac{\partial J_{ui}}{\partial p_u} = -2(r_{ui} - \hat{r}_{ui})q_i + 2\lambda p_u = -2\epsilon_{ui}q_i + 2\lambda p_u$
- $\frac{\partial J_{ui}}{\partial q_i} = -2(r_{ui} - \hat{r}_{ui})p_u + 2\lambda q_i = -2\epsilon_{ui}p_u + 2\lambda q_i$
- $\frac{\partial J_{ui}}{\partial b_u} = -2(r_{ui} - \hat{r}_{ui}) + 2\lambda b_u = -2\epsilon_{ui} + 2\lambda b_u$
- $\frac{\partial J_{ui}}{\partial b_i} = -2(r_{ui} - \hat{r}_{ui}) + 2\lambda b_i = -2\epsilon_{ui} + 2\lambda b_i$

2. Regras de Atualização

Seguindo a direção oposta ao gradiente ($\theta \leftarrow \theta - \eta\nabla J$) e absorvendo o fator 2 na taxa de aprendizagem η, obtemos as seguintes regras de atualização:

- $p_u \leftarrow p_u + \eta(\epsilon_{ui}q_i - \lambda p_u)$
- $q_i \leftarrow q_i + \eta(\epsilon_{ui}p_u - \lambda q_i)$
- $b_u \leftarrow b_u + \eta(\epsilon_{ui} - \lambda b_u)$
- $b_i \leftarrow b_i + \eta(\epsilon_{ui} - \lambda b_i)$

T3 (Pairwise gradient)

O objetivo é demonstrar que o gradiente da função de perda logística em relação aos parâmetros ϕ é dado por:

$$\nabla_\phi \left(-\log \sigma(\phi^\top \Delta f)\right) = \left(\sigma(\phi^\top \Delta f) - 1\right) \Delta f \tag{10}$$

Demonstração Passo a Passo

Seja $z = \phi^\top \Delta f$. A função que queremos derivar é $L(\phi) = -\log \sigma(z)$.

1. Derivada da função sigmóide (σ): Sabemos que a derivada da função sigmóide $\sigma(z) = \frac{1}{1+e^{-z}}$ em relação a z é:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{11}$$

2. Aplicação da Regra da Cadeia: Utilizando a regra da cadeia para o logaritmo e para a sigmóide:

$$\nabla_\phi L(\phi) = -\frac{1}{\sigma(z)} \cdot \frac{\partial \sigma(z)}{\partial z} \cdot \nabla_\phi z \tag{12}$$

3. Substituição das derivadas: Substituindo $\sigma'(z)$ e notando que $\nabla_\phi(\phi^\top \Delta f) = \Delta f$:

$$\nabla_\phi L(\phi) = -\frac{1}{\sigma(z)} \cdot [\sigma(z)(1 - \sigma(z))] \cdot \Delta f \tag{13}$$

4. **Simplificação:** Os termos $\sigma(z)$ no numerador e denominador anulam-se:

$$\nabla_\phi L(\phi) = -(1 - \sigma(z))\Delta f \tag{14}$$

$$\nabla_\phi L(\phi) = (\sigma(z) - 1)\Delta f \tag{15}$$

5. **Resultado Final:** Substituindo z de volta pela expressão original $\phi^\top \Delta f$:

$$\nabla_\phi \left(-\log \sigma(\phi^\top \Delta f)\right) = \left(\sigma(\phi^\top \Delta f) - 1\right) \Delta f \tag{16}$$

T4 (Why RMSE $\neq$ ranking quality?)

Otimizar o RMSE (Root Mean Squared Error) em modelos de Fatoração de Matrizes nem sempre garante uma boa qualidade de recomendação Top-K devido aos seguintes fatores:

- **Foco na Regressão vs. Ordenação:** O RMSE trata o problema como uma regressão pontual, tentando prever o valor exato de cada classificação. No entanto, para recomendações, o que importa é a ordem relativa entre os itens (ranking) e não a precisão do valor absoluto da nota.
- **Peso Igual a Erros Irrelevantes:** O RMSE penaliza igualmente um erro de previsão num item que o utilizador detesta (ex: prever 2 em vez de 1) e num item que o utilizador adoraria (ex: prever 4 em vez de 5). Para o cenário de Top-K, apenas os erros ocorridos no topo da lista são realmente críticos para a experiência do utilizador.
- **Desalinhamento com Métricas de Negócio:** O objetivo de um sistema de entretenimento é frequentemente maximizar métricas como NDCG ou Recall, que dependem da posição dos itens relevantes na lista. Um modelo pode reduzir o seu RMSE global ao prever melhor as notas baixas da "cauda longa"(long-tail) sem nunca melhorar a visibilidade dos itens que o utilizador realmente escolheria consumir.
- **Viés de Dados Observados:** O RMSE é calculado apenas sobre interações observadas (ratings explícitos). Isto ignora a estrutura de interações implícitas, onde o ato de escolher um item entre várias opções num conjunto de candidatos é um sinal de preferência muito mais forte para o ranking do que a precisão de uma nota isolada.

Observações e Resultados do Projeto

No âmbito do projeto desenvolvido, observou-se que a redução do RMSE durante o treino do MF-ALS não se traduziu numa melhoria equivalente das métricas de ranking. Em particular, apesar de o RMSE descer de forma consistente, o NDCG@10 médio manteve-se baixo (0.0297), enquanto o modelo pairwise LTR, que otimiza diretamente a ordenação, atingiu NDCG@10 de 0.0470. Este contraste confirma que otimizar o erro de previsão de ratings não garante uma boa ordenação dos itens no topo da lista.

T5 (Offline evaluation caveat)

A avaliação offline pode ser enganosa porque os dados históricos não são neutros, mas sim o resultado de uma política de exposição anterior que gera um **viés de seleção**. Itens que nunca foram mostrados constituem "pontos cegos", impossibilitando testar a reação do utilizador a esses conteúdos. Além disso, o modelo pode limitar-se a imitar os enviesamentos do sistema antigo, ignorando a **natureza de ciclo fechado** dos recomendadores, onde o ranking atua como uma intervenção que molda o comportamento e

os dados futuros. Sem exposição aleatória, criam-se **ciclos de feedback** que amplificam a popularidade e reduzem a diversidade, algo que métricas estáticas como o NDCG não conseguem capturar totalmente por ignorarem a "deriva de dados"(data drift) causada pelo próprio sistema.